Technique: Adversary argument

- An adversary can be thought of as a very clever villain or demon. It is represented in talk slides as a cute cartoon devil, for example: 
- Murphy's Law: "Anything that can go wrong will go wrong" It is the adversary's job to make sure that this happens.
- The fun part: you get to be the adversary!

How it works

- The algorithm makes value queries, and the model says that the algorithm will receive a response. But we allow the adversary to figure out what that response should be!
- The goal of the adversary is to respond to queries in such a way that the algorithm makes slow progress, which results in a high cost.
- How is this possible? We'll run the algorithm without a pre-defined input, and the adversary makes up the input as it goes along.

Definition of an Adversary for the Guessing Game

- The initial range of possible values for x is 1,…,n, but x has no initial value.
- The algorithm performs a value query “is x = i?”, and the adversary has to provide a response. To keep the algorithm running as long as possible, the adversary wants to keep responding “x < i” or “x > i”
- So the adversary keeps track of a range of values for x that are consistent with all of the responses it has given so far. We’ll call it min,…,max.
  Initially, all values are possible, so min=1 and max=n
- When the algorithm queries “is x = i?”, the adversary chooses the response that keeps the range min,…,max as large as possible.

Definition of an Adversary for the Guessing Game

- If i=min=max, then respond “x=i”. Otherwise:
- If i < min, then min and max are unchanged, the adversary responds “x > i”
- If i > max, then min and max are unchanged, the adversary responds “x < i”
- If i is in the range min,…,max, then the adversary compares the two ranges min,…,i-1 and i+1,…max.

  If the first is strictly larger, then respond “x < i” and set max = i-1

  else, respond “x > i” and set min = i+1 (Note: when both subranges have equal length, this action is always taken)

Summary

- Summary of the adversary's strategy:
  - Keep as many values for x "in play" as possible.
  - As the algorithm performs queries, the range of possible values might shrink, but the adversary responds to the query with the response that minimizes the number of values eliminated.
  - By doing this, the adversary slows the algorithm down and forces it to perform more queries.

Example with range 1,…8

- Query 1: “x=3?”
  Adversary: “x > 3” (since {4,…,8} is larger than {1,2})
- Query 2: “x=1?”
  Adversary: “x > 1” (since our valid range is {4,…,8})
- Query 3: “x=6?”
  Adversary: “x > 6”
  (range {4,5} is not strictly larger than {7,8}, so {7,8} is the new range)
- Query 4: “x=8?”
  Adversary: “x < 8” (range {7} is larger than range {})
- Query 5: “x=7?”
  Adversary: “x=7”

Sounds like cheating

- In a sense, it is cheating. We're running the algorithm without an actual input defined in advance.
- But, more importantly, it's not cheating as long as all of the responses that the adversary gives are consistent with one possible input!
- Why? Because we can just run the algorithm again with that input (and no adversary), and the exact same sequence of query responses will be given.
  E.g., on previous slide, re-run algorithm with x=7 defined at the beginning.
  - An indistinguishability argument shows that the exact same execution would occur both times. (Practice: try writing this proof. Make sure to define histories for this new model)

Proof Structure

- To write out the proof, you would start by giving a full definition of an adversary. (See slides titled "Definition of an Adversary for the Guessing Game")
- Argue the correctness of the adversary: at the end, it will construct a valid input that is consistent with the answers it gave to the algorithm's queries.
- Then, you would argue about the number of queries that occur before the adversary will answer "x=i". For the adversary I defined in these slides, it's how many queries until min=max.
    - In particular, you would prove that the size of the adversary's range decreases by at most half with each query (justification: because the adversary always chooses the larger sub-range to keep)
- It follows that $\Omega(\log_2 n)$ queries are necessary.

  (There are some floors/ceilings if n isn't a power of 2, but not a huge deal)

Problem: Find the maximum and minimum element

- Task: output the two indices where the maximum and minimum can be found.
- First, let's think about some algorithms.
- The most obvious is just mergesort the array in ascending order and output 0 and n-1. This has cost O(n log n).
- Next obvious idea is to
  1. Find the maximum, put it at X[n-1]
  2. Then find the minimum among X[0..n-2].
     - This has cost (n-1)+(n-2) = 2n-3 using the "Scan Algorithm" (and a simple variant for minimum) from earlier.
- Can you think of something more clever? or is 2n-3 the optimal cost?

Lower Bound Idea

- As before, it's not obvious how to do better, so let's see if we can find a matching lower bound.
- Let's think about winners and losers again.
- Initially, no element has won or lost, so let's put them in a set called "Unknowns".
- The Unknowns set will be the elements that the algorithm knows nothing about. Any of them could be the maximum or minimum.

Lower Bound Idea

- Once an element e is involved in a comparison, the algorithm has gained some information about it.
  - if e was a winner, it's possible to conclude that e can't be the minimum
  - if e was a loser, it's possible to conclude that e can't be the maximum
  - We can't assume that the algorithm will come to these conclusions! But they are motivating the idea behind the proof.
- So let's make two more sets:
  1. Winners: elements that have won at least once, and have never lost
  2. Losers: elements that have lost at least once, and never won

Lower Bound Idea

- What about elements that have lost and won? They are pretty useless, and a smart algorithm might conclude that, so let's put them in a set called Useless.
- To recap, we have Unknowns, Winners, Losers, Useless. Looking at their definitions, we see that each element must belong to exactly one of the four sets at any given time.

Lower Bound Idea

- Let's make some observations about what an algorithm must accomplish:
  - At termination, the Unknowns set must be empty. Why?
    - This is because an element that hasn't been in a comparison could be the maximum or minimum. (Use an indistinguishability proof)
  - At termination, the Winners and Losers sets must have exactly one element each. Why?
    - Again, by indistinguishability, if there are two elements that never lost, either can be the maximum. Similarly, either of two elements that never won could be the minimum.
- So we define an adversary to slow down progress towards these accomplishments.

Definition of the Adversary

- The algorithm chooses a comparison (i,j), and the adversary looks at which sets contains them.
- If exactly one element is in Winners, then make that element the winner of the comparison.
- If exactly one element is in Losers, then make that element the loser of the comparison.
- (These two cases prevent elements from leaving the Winners and Losers sets, which slows down progress.)

Definition of the Adversary

- When both elements are in Unknowns, both in Winners, or both in Losers: it doesn't matter what we say, just answer X[i] > X[j]
- However, when an element moves from Losers to Useless, assign it the largest negative integer that hasn't been assigned yet.
- When an element moves from Winners to Useless, assign it the smallest positive integer that hasn't been assigned yet.
- When both elements are in Useless, answer consistently using the integers we assigned when they were moved to Useless.

Analysis

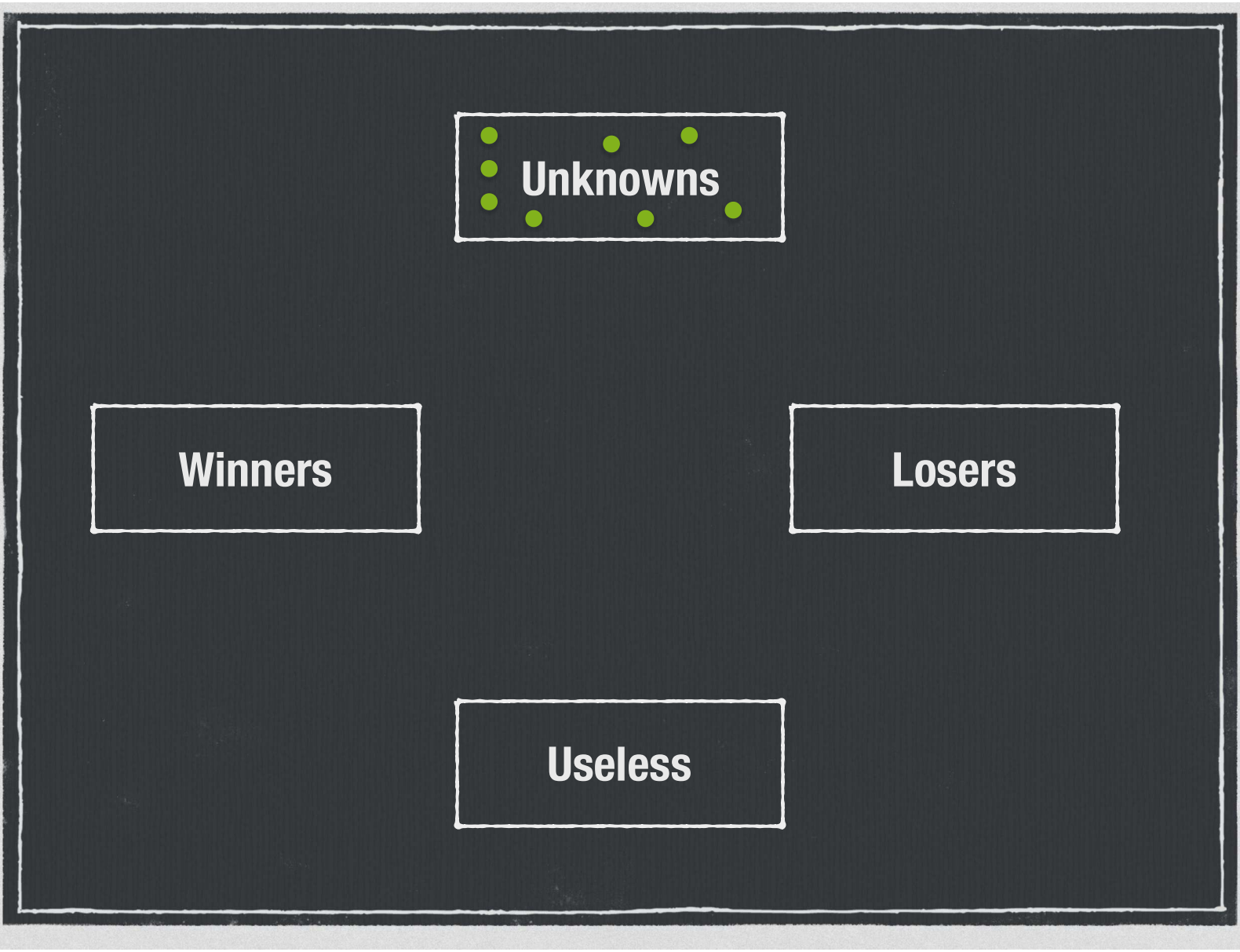
- So we've defined an adversary on the previous two slides. What remains is to analyze how many comparisons it forces the algorithm to perform.
- Initially, we have the following set sizes:
  - Unknowns = n
  - Winners = Losers = Useless = 0
- From our earlier observation, when the algorithm terminates:
  - Unknowns = 0
  - Winners = 1, Losers = 1
  - Useless = n-2

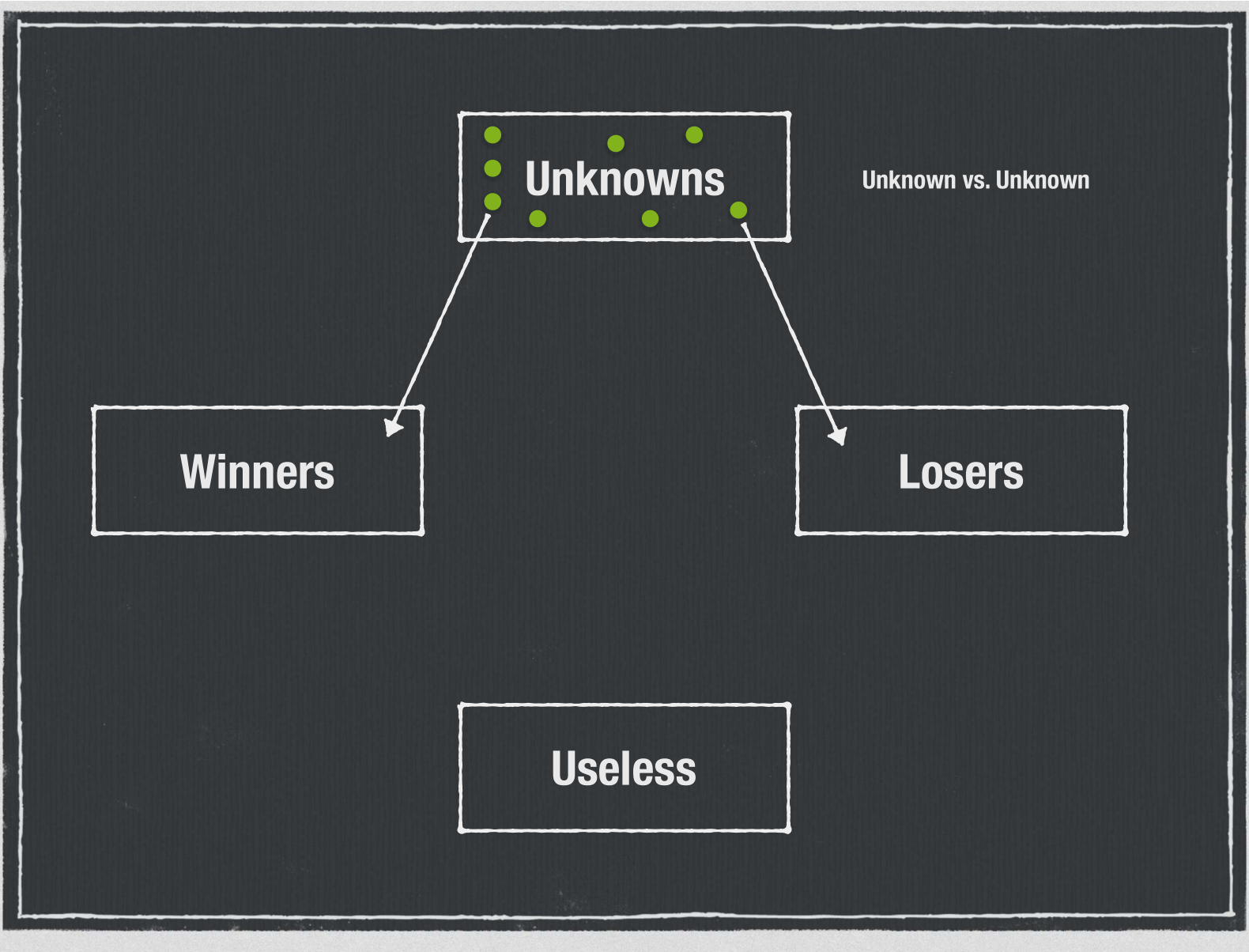
Analysis

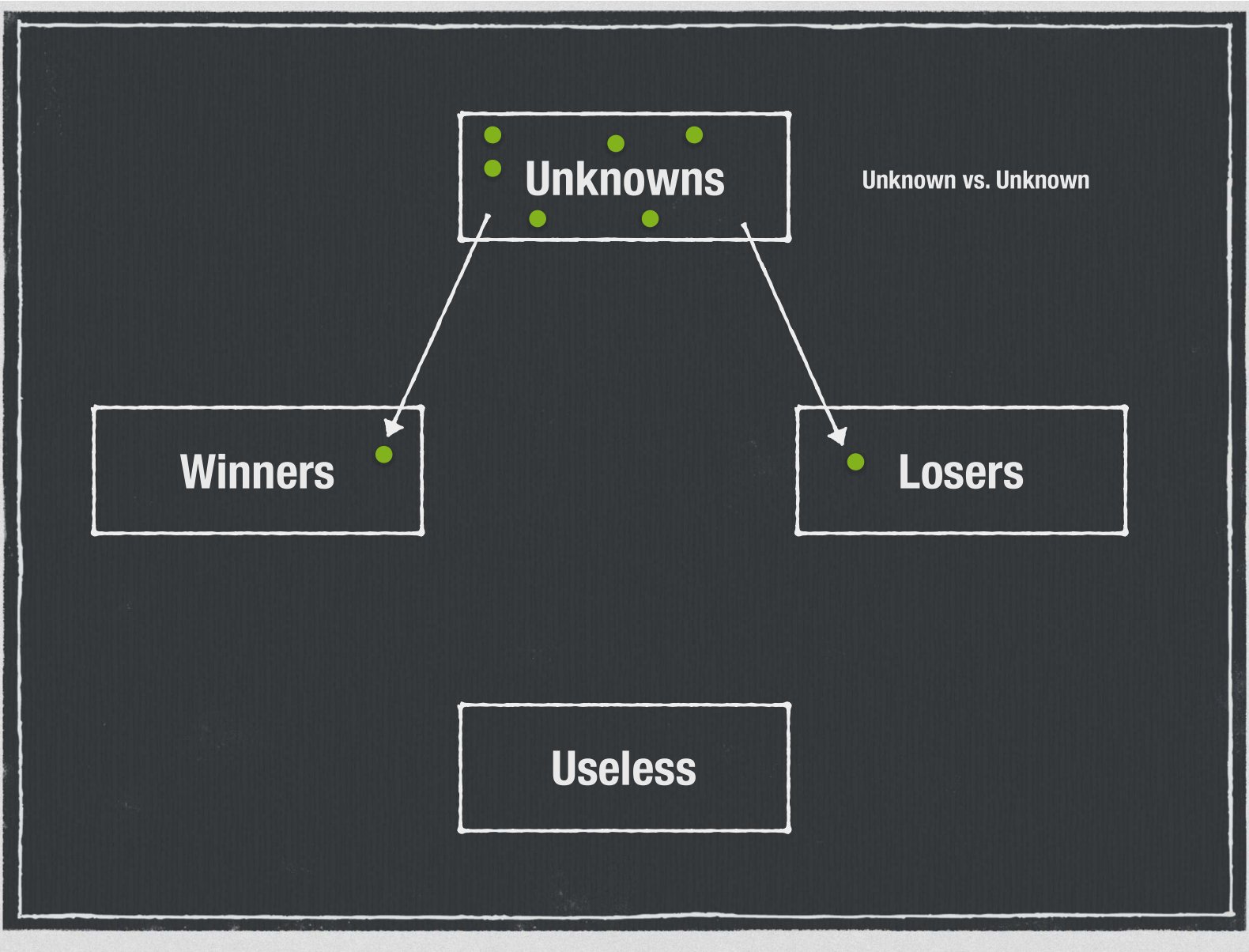
- Initially, we have the following set sizes:
  - Unknowns = n
  - Winners = Losers = Useless = 0
- From our earlier observation, when the algorithm terminates:
  - Unknowns = 0
  - Winners = 1, Losers = 1
  - Useless = n-2
- Notice: an element e cannot move directly from Unknowns to Useless.

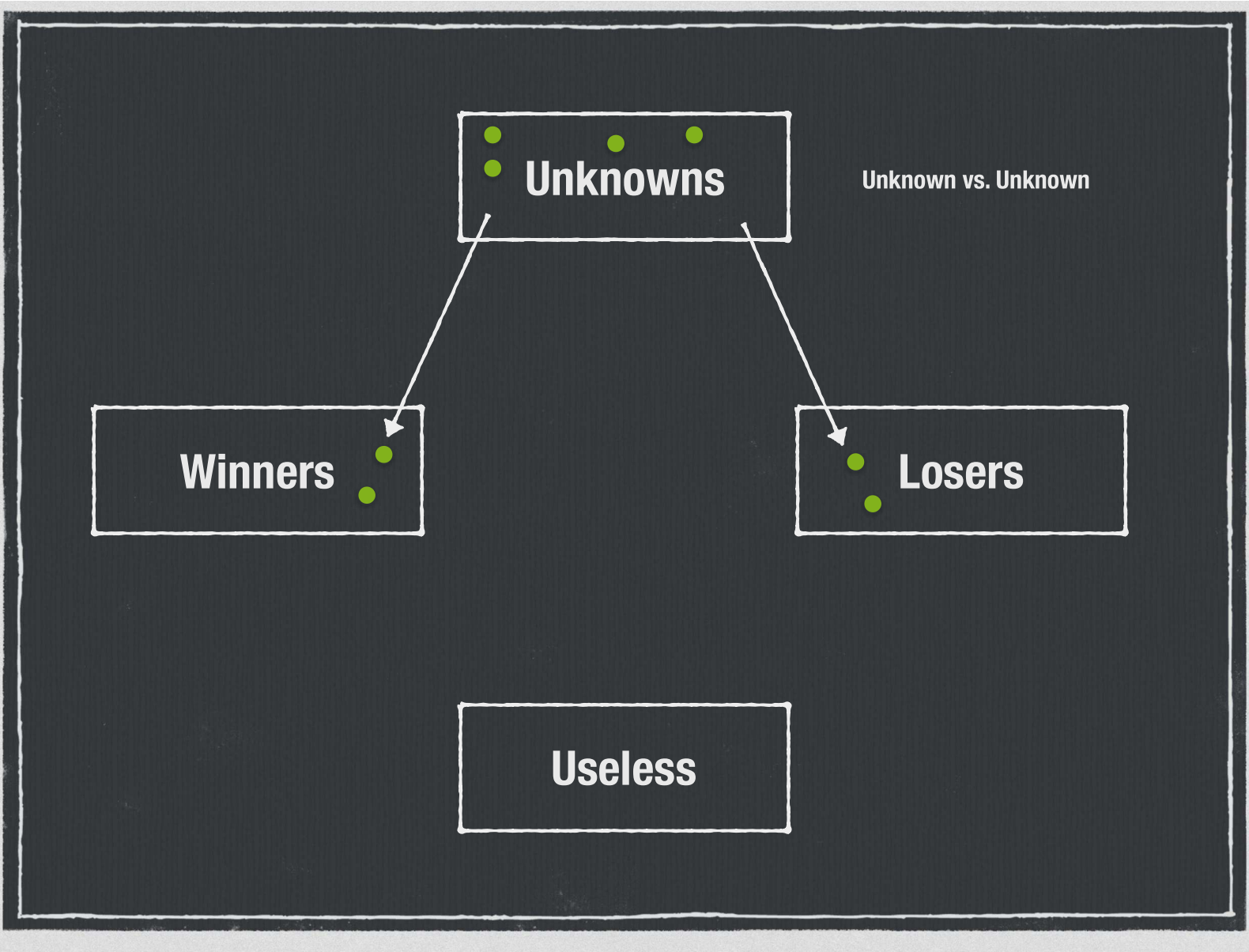
  The first comparison involving e moves it to Winners or Losers.
- So each element goes from

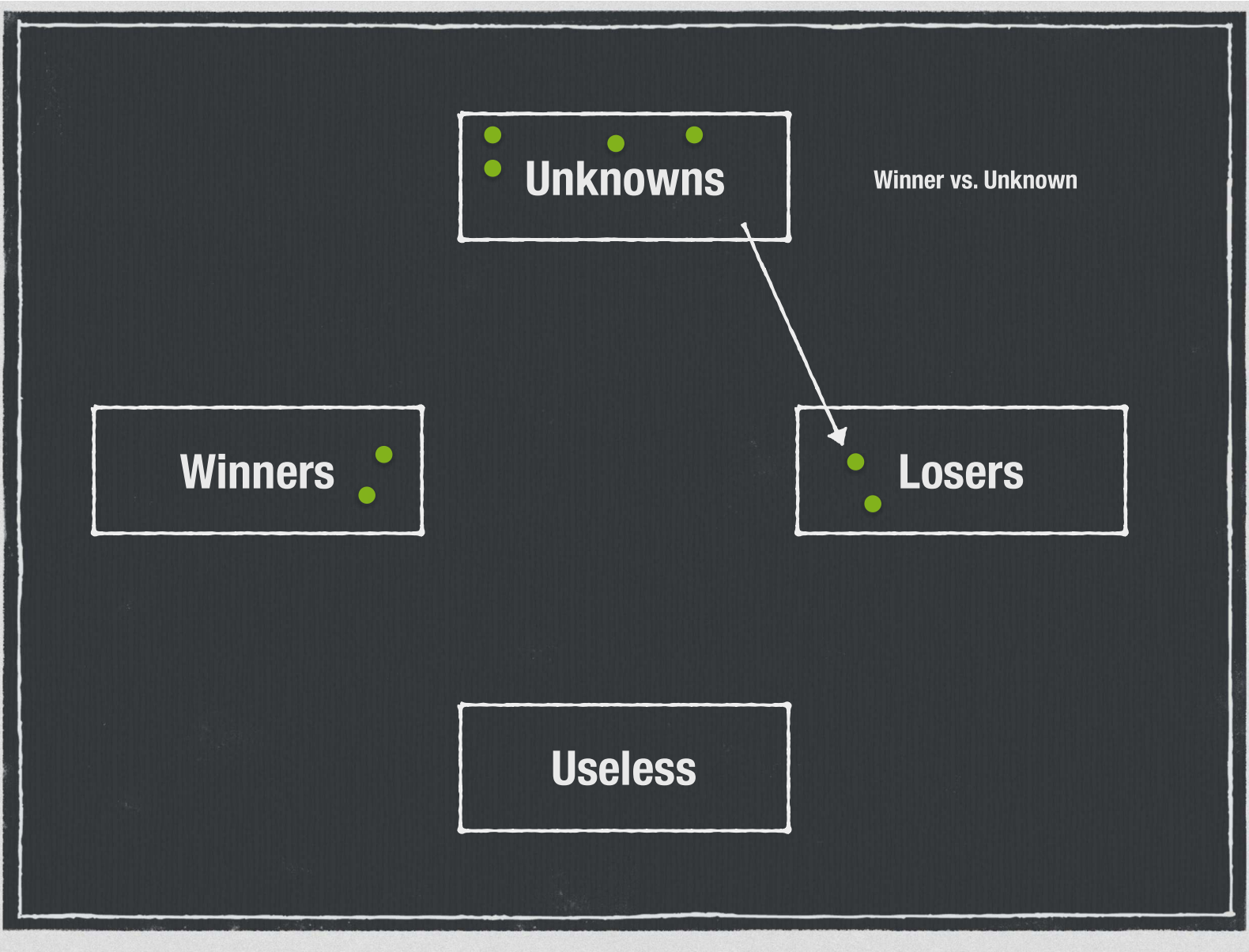
  Unknowns $\longrightarrow$ Winners $\cup$ Losers $\longrightarrow$ Useless

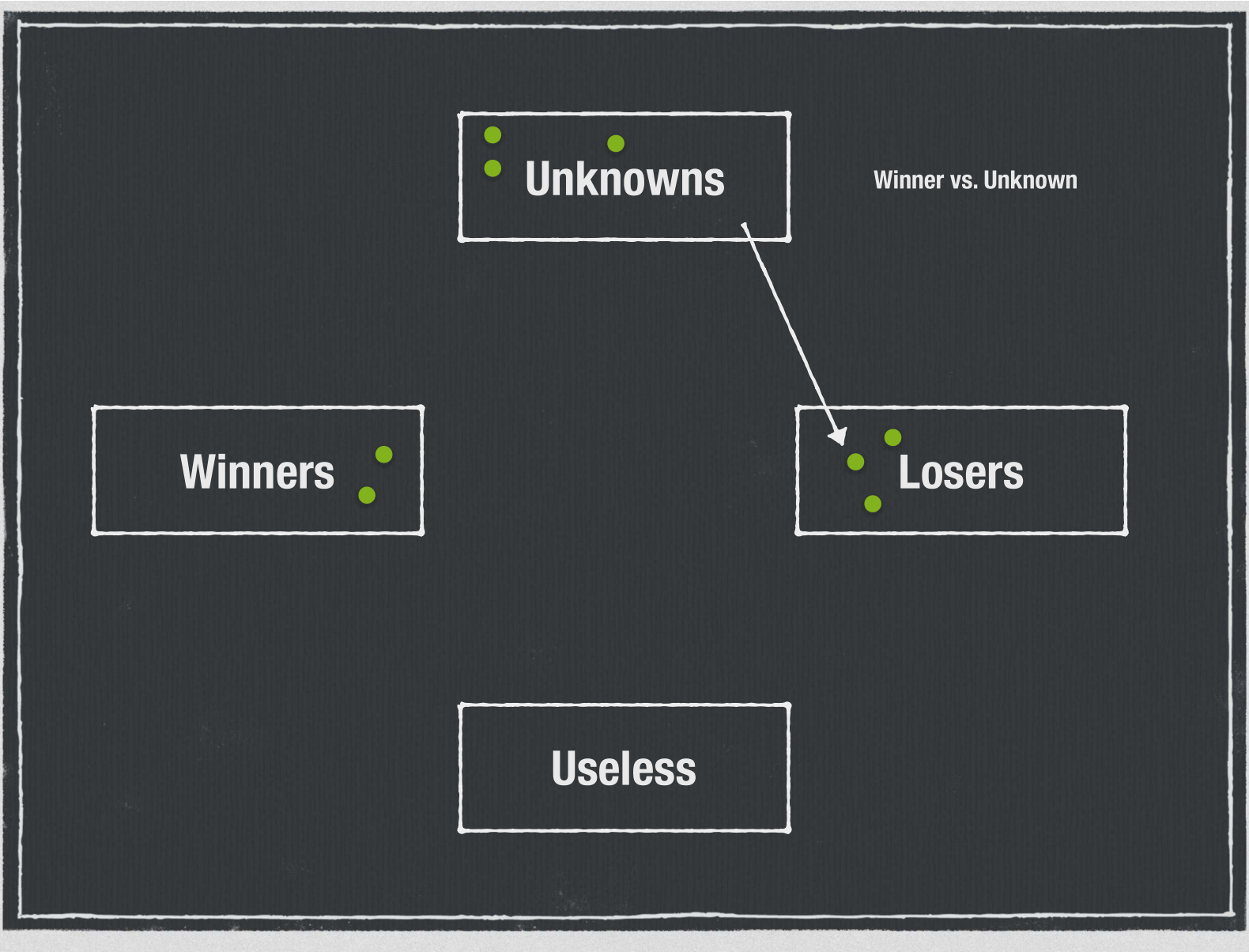

Unknowns
Winners
Losers
Useless

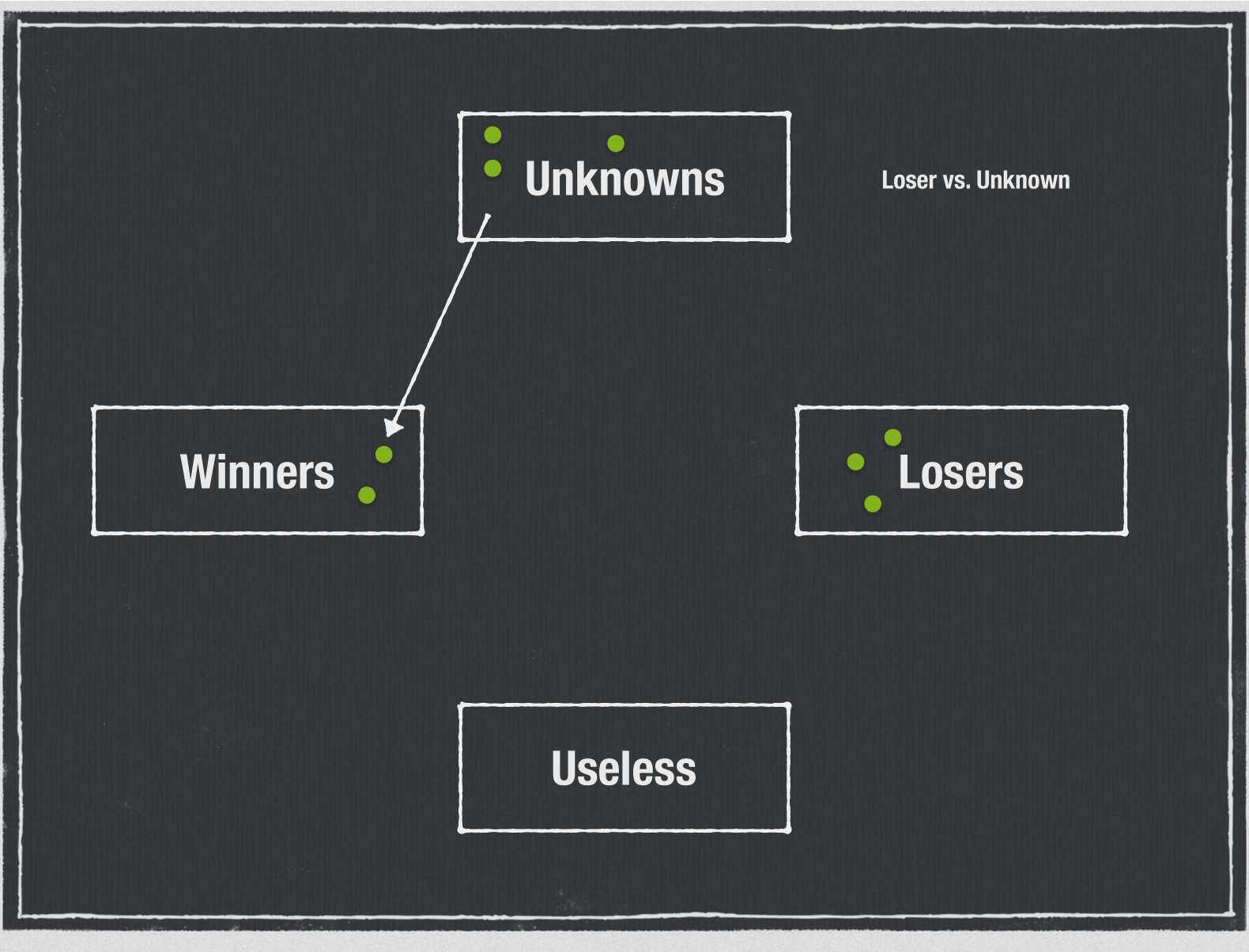

Unknowns
Unknown vs. Unknown
Winners
Losers
Useless

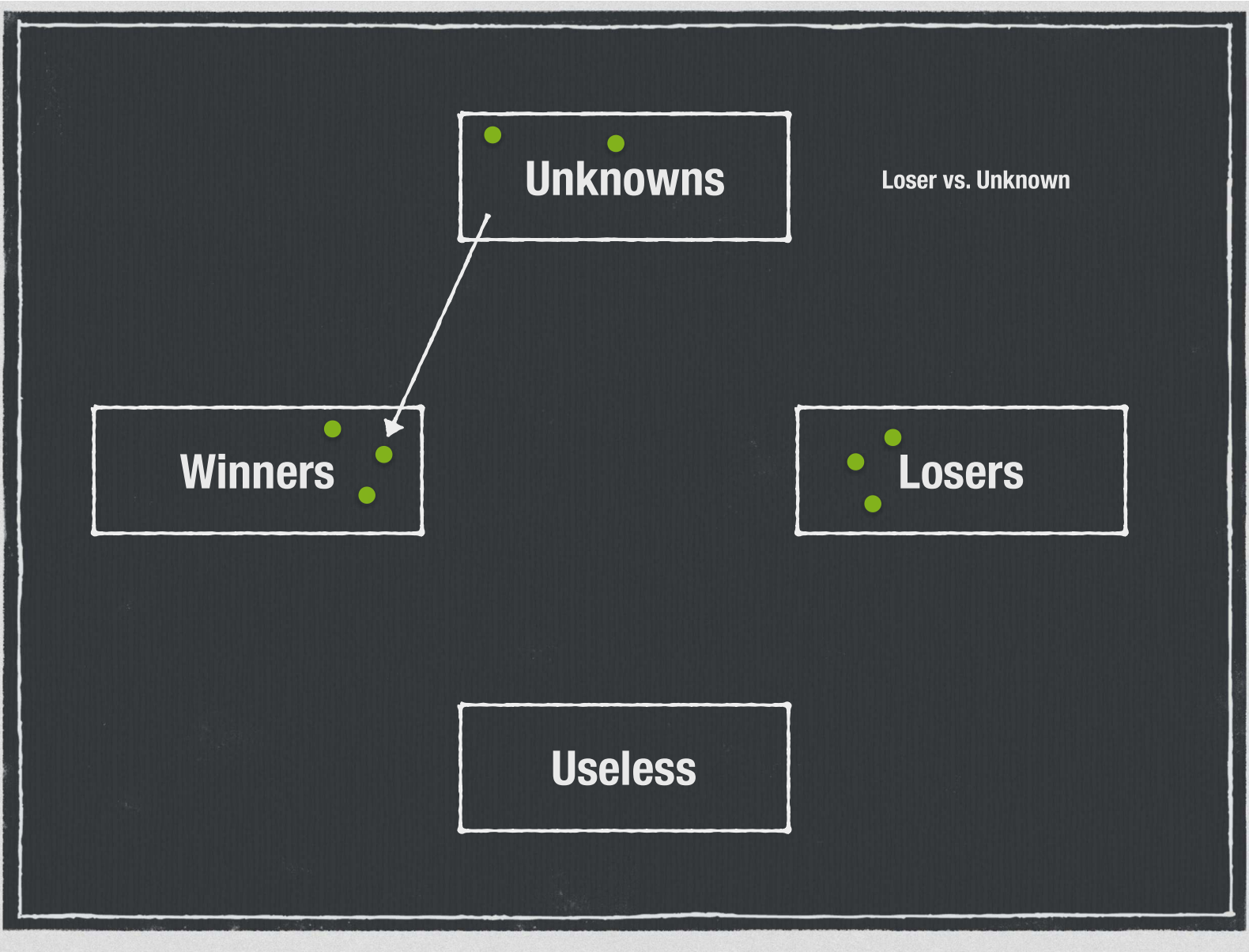

Unknowns
Unknown vs. Unknown
Winners
Losers
Useless

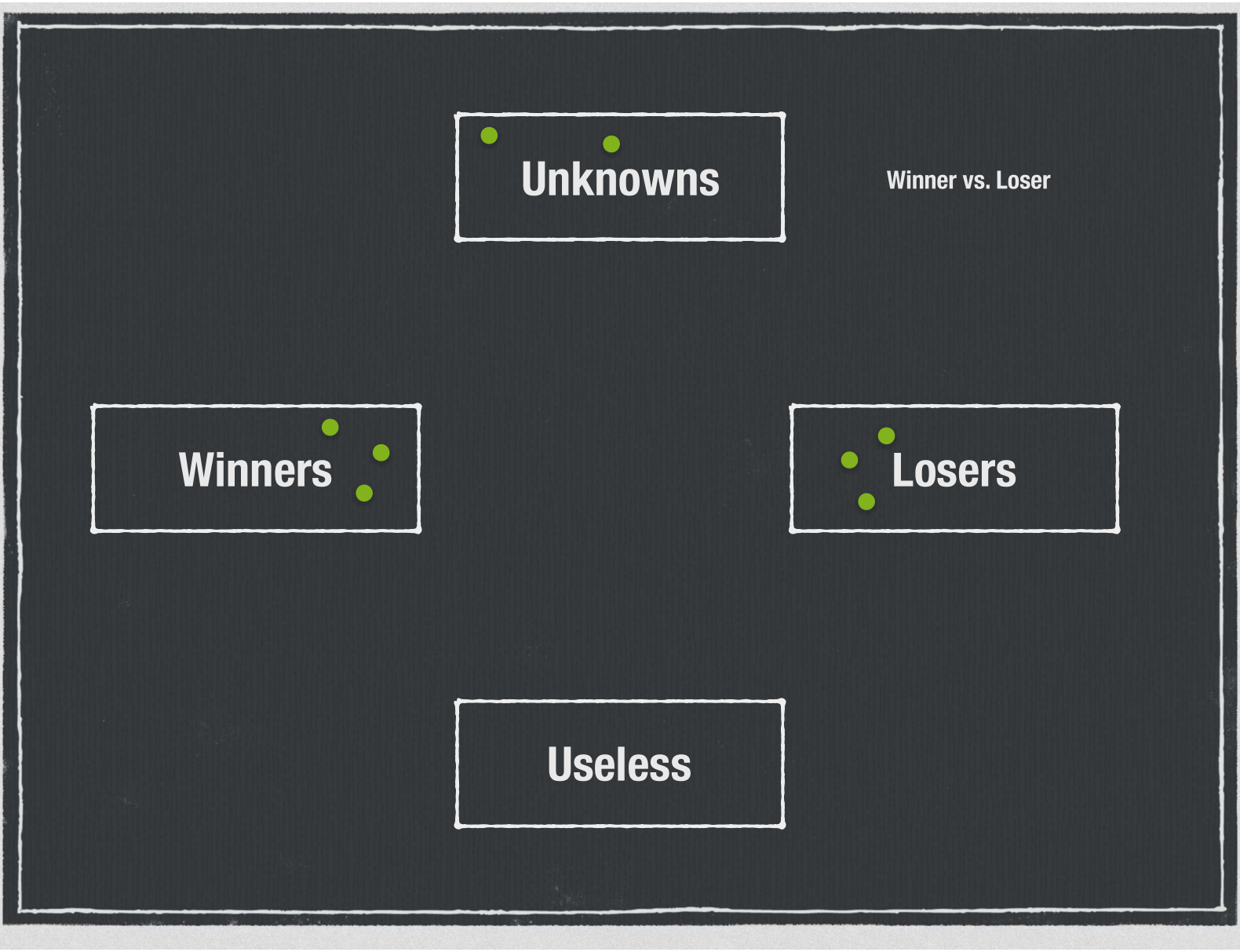

Unknowns
Unknown vs. Unknown
Winners
Losers
Useless

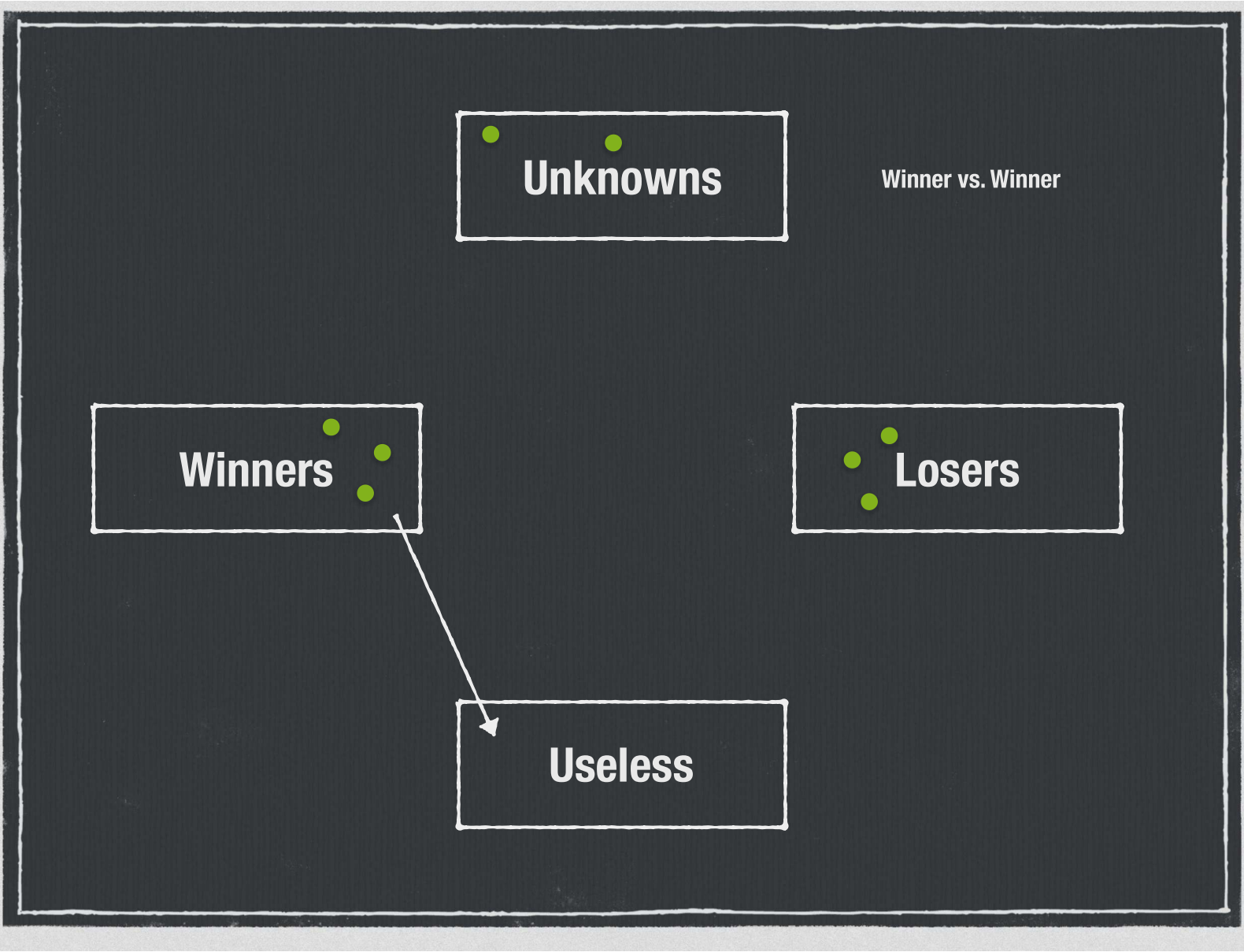

Unknowns
Winner vs. Unknown
Winners
Losers
Useless

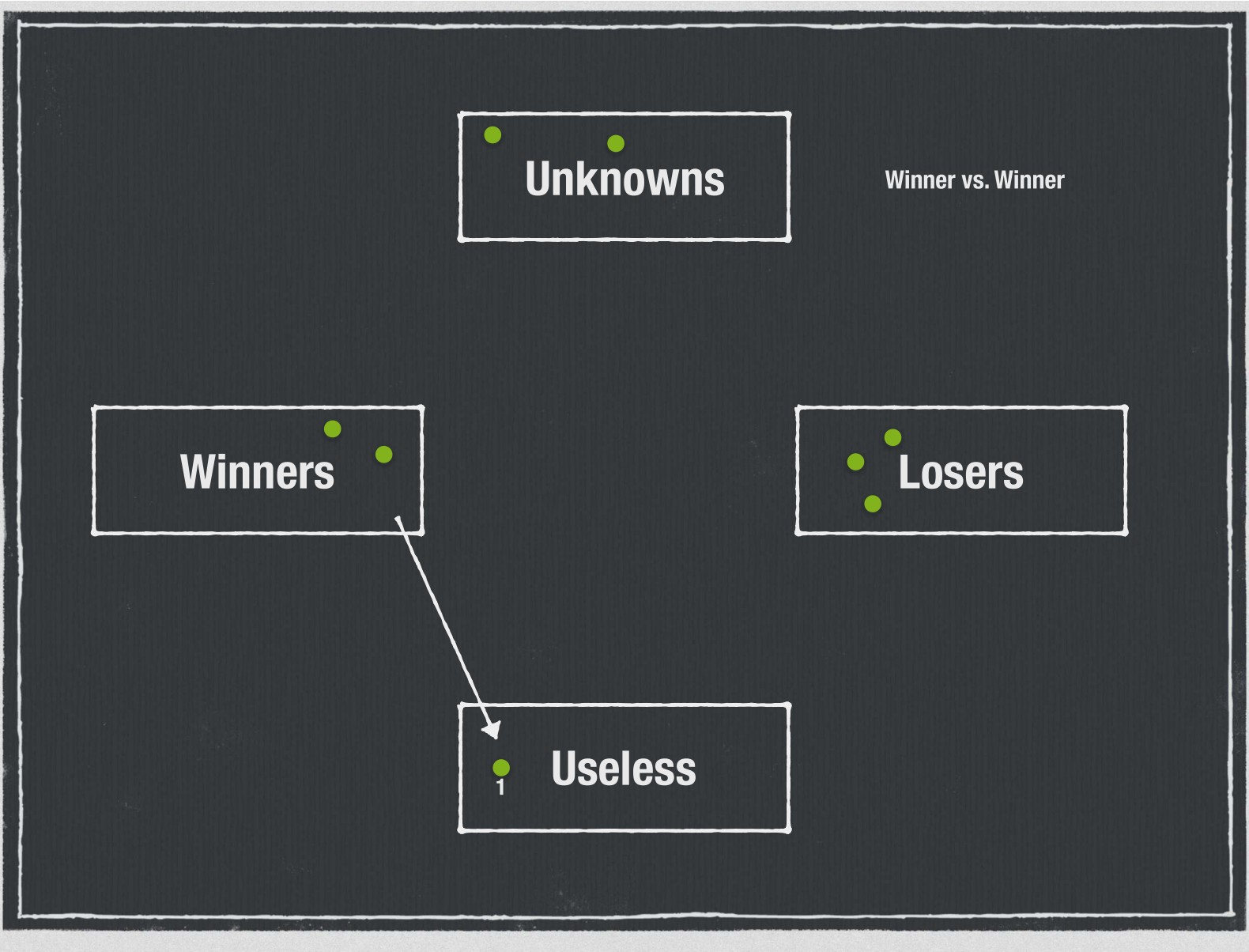

Unknowns
Winner vs. Unknown
Winners
Losers
Useless

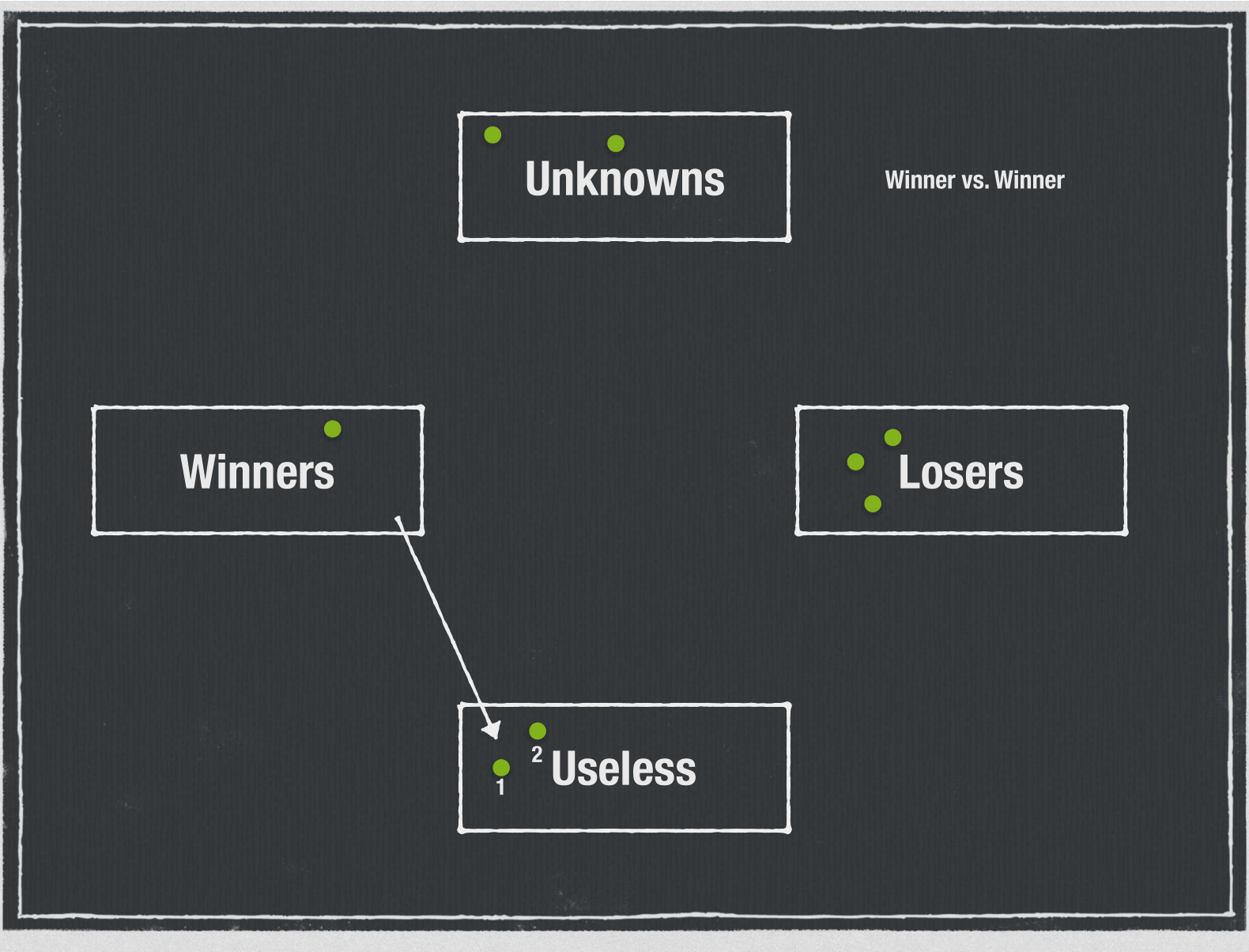

Unknowns
Loser vs. Unknown
Winners
Losers
Useless

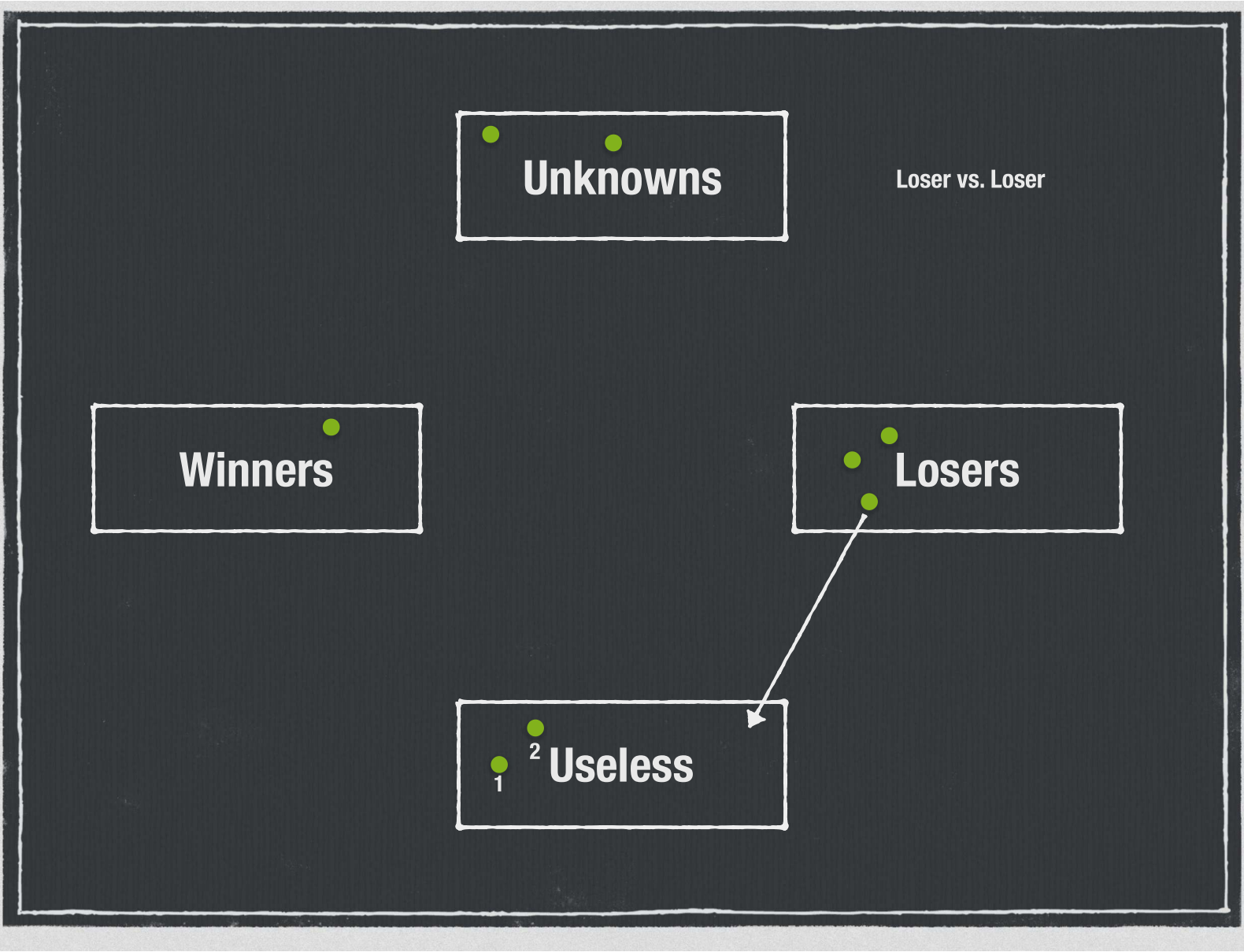

Unknowns
Loser vs. Unknown
Winners
Losers
Useless

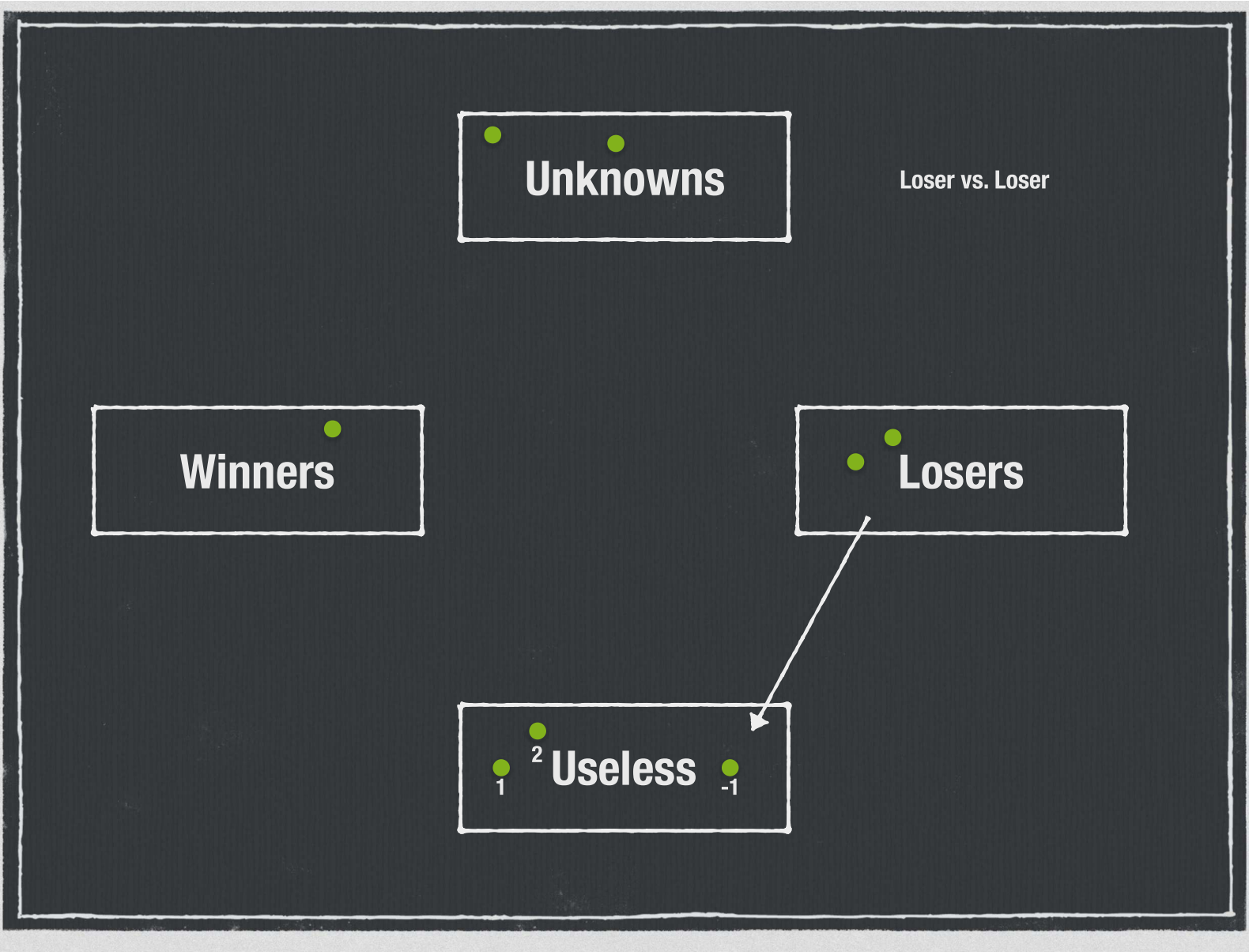

Unknowns
Winner vs. Loser
Winners
Losers
Useless


Unknowns
Winner vs. Winner
Winners
Losers
Useless

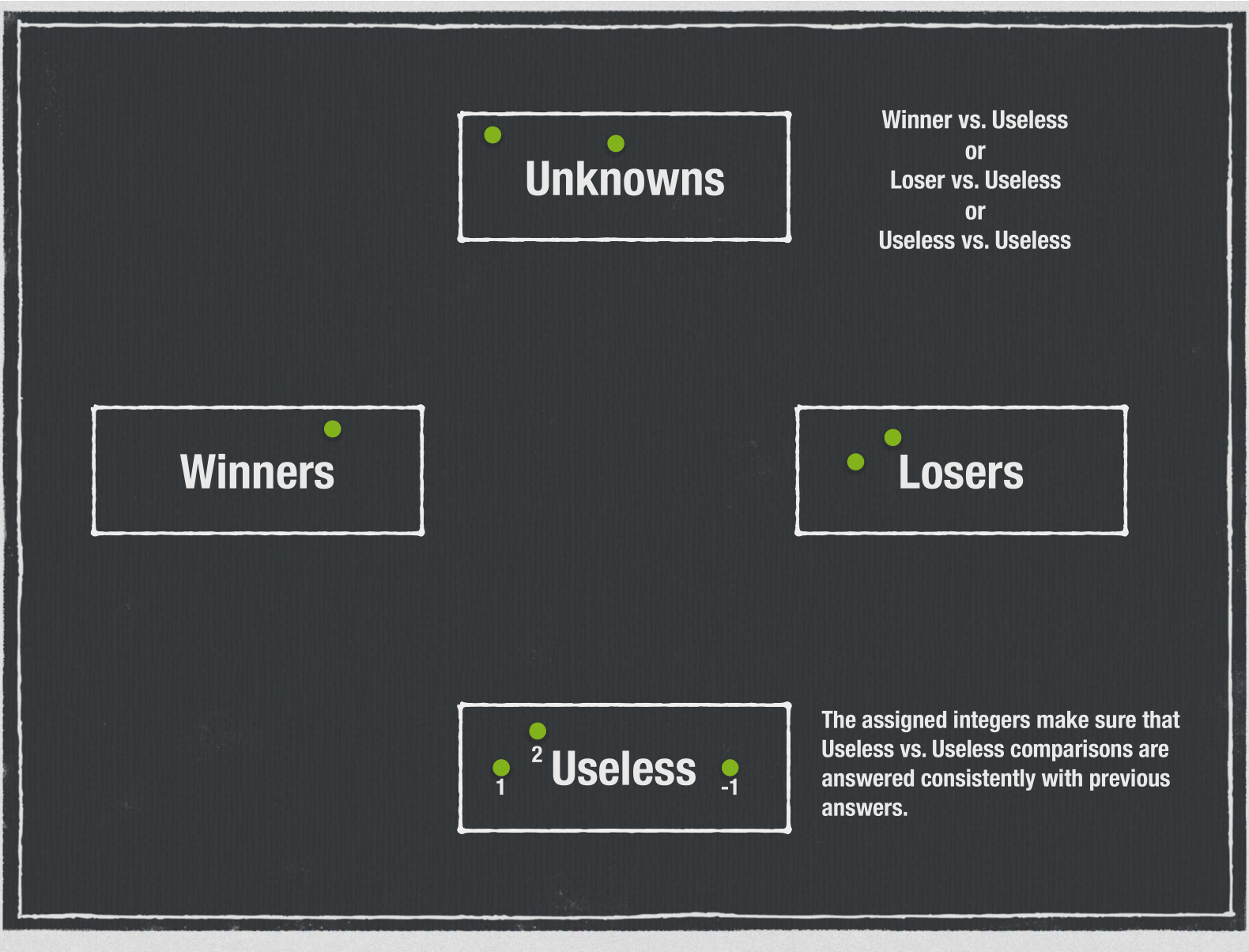

Unknowns
Winner vs. Winner
Winners
Losers
Useless
1

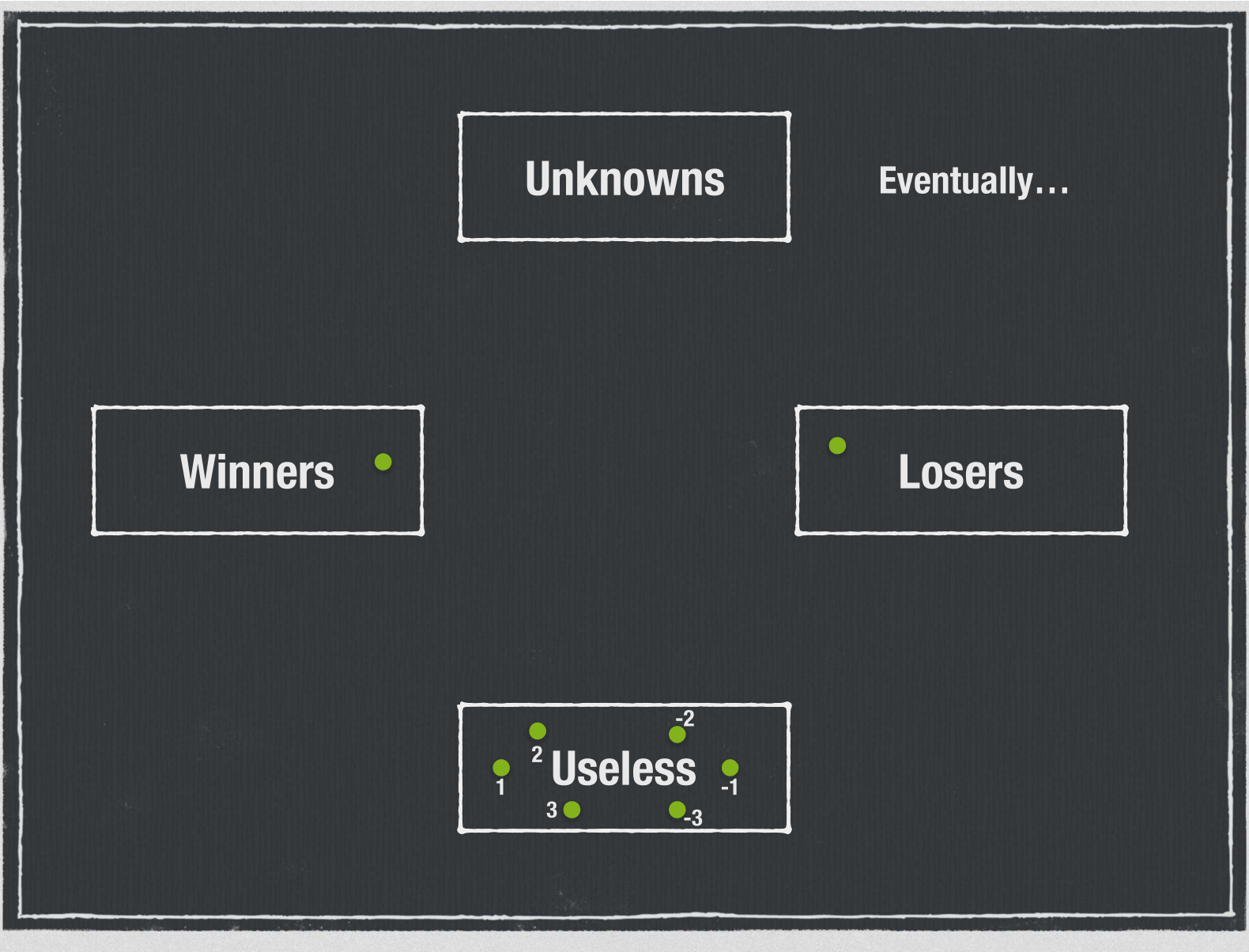

Unknowns
Winner vs. Winner
Winners
Losers
2
Useless
1


Unknowns
Loser vs. Loser
Winners
Losers
2
Useless
1


Unknowns
Loser vs. Loser
Winners
Losers
2
Useless
1
-1


Unknowns
Winner vs. Useless
or
Loser vs. Useless
or
Useless vs. Useless
Winners
Losers
2
Useless
1
-1


Unknowns
Winner vs. Useless
or
Loser vs. Useless
or
Useless vs. Useless
Winners
Losers
2
1
Useless
-1
The assigned integers make sure that Useless vs. Useless comparisons are answered consistently with previous answers.


Unknowns
Eventually…
Winners
Losers
-2
2
Useless
1
-1
3
-3

Analysis

- Eventually all n elements must leave Unknowns, and no more than two elements can leave Unknowns in a single comparison.
  - So at least ⌈n/2⌉ comparisons are required. All of these comparisons involve elements leaving Unknowns and going to Winners ∪ Losers.
- n-2 elements must move from (Winners ∪ Losers) to Useless.
  - Referring to the adversary's strategy (and diagrams on previous slides), this only happens when we compare two elements in Winners or two elements in Losers. As a result, only one element moves to Useless with each such comparison.
- Adding these two up, we see that at least ⌈n/2⌉+(n-2) comparisons must occur, which gives ⌈3n/2⌉ - 2.

Matching Upper Bound:
An algorithm using $\lceil 3n/2 \rceil - 2$ comparisons

- Thanks to this lower bound proof, an optimal algorithm pretty much falls into our lap.
- Do exactly what the proof did: define sets Unknowns, Winners, Losers, Useless.

  Initially, all array elements are considered Unknowns.

  1. While (# of Unknowns) ≥ 2, compare two elements from Unknowns.

     Put the larger one in Winners and put the smaller one in Losers.
  2. If (# of Unknowns) == 1, compare the e ∈ Unknown with any other element.

     If e is smaller, put in Losers. If e is bigger, put in Winners. (this is special case if n is odd)
  3. While (# of Losers) ≥ 2, compare two elements from Losers. Put bigger one in Useless.
  4. While (# of Winners) ≥ 2, compare two elements from Winners. Put smaller one in Useless.
  5. Output the last x in Losers as min, output the last y in Winners as max.